

Nuclear Signal Quantification

Nuclear segmentation, Nuclear signal measurements
& Cell classification from confocal images

Denis Lucas

September 2026

Contents

1	Pipeline Overview	3
1.1	Objective	3
1.2	Files structure	3
2	Scripts description	4
2.1	config.yaml	4
2.2	config.py	4
2.3	01_rename_files.py	4
2.3.1	Objective	4
2.3.2	Input	4
2.3.3	Process	4
2.3.4	Output	4
2.4	02_sort_Airy_RAW.py	5
2.4.1	Objective	5
2.4.2	Input	5
2.4.3	Process	5
2.4.4	Output	5
2.5	03_split_czi.py	5
2.5.1	Objective	5
2.5.2	Input	5
2.5.3	Process	5
2.5.4	Output	5
2.6	04_segmentation.py	6
2.6.1	Objective	6
2.6.2	Input	6
2.6.3	Process	6
2.6.4	Output	6
2.6.5	04b_control_segmentation.py	6
2.7	05_quantification.py	7
2.7.1	Objective	7
2.7.2	Input	7
2.7.3	Process	7
2.7.4	Output	7

2.7.5	05b_plot_intensity_distribution.py	7
2.8	06_threshold.py	8
2.8.1	Objective	8
2.8.2	Input	8
2.8.3	Process	8
2.8.4	Output	8
2.9	07_classification.py	9
2.9.1	Objective	9
2.9.2	Input	9
2.9.3	Process	9
2.9.4	Output	9
2.9.5	07b_control_classification.py	9
2.9.6	07c_plot_thresholds.py	10
2.10	08_plot_quantif.py	11
2.10.1	Objective	11
2.10.2	Input	11
2.10.3	Process	11
2.10.4	Output	11
3	Installation	12
4	How to use the pipeline	12
4.1	Run a script	12
4.2	Pipeline parameters	12
5	Author and contact	12

1 Pipeline Overview

1.1 Objective

This pipeline is designed to quantify the expression of nuclear markers from immunolabeled confocal microscopy images and to classify individual nuclei into positive and negative populations based on marker expression levels.

The pipeline consists of the following main steps:

1. **Data preparation:** preprocessing and organization of the input data.
2. **Nuclei segmentation:** identification and segmentation of individual nuclei from the microscopy images.
3. **Signal quantification:** measurement of the nuclear signal intensity for each marker.
4. **Threshold selection:** determination of the intensity threshold used to distinguish marker-positive from marker-negative nuclei.
5. **Cell classification and quantification:** classification of individual nuclei into positive and negative populations and calculation of the proportion of cells in each population.

1.2 Files structure

The general structure of the project is as follows:

```
Quantification/  
|  
|-- README.pdf  
|  
|-- config.yaml  
|  
|-- scripts/  
|   |-- config.py  
|   |-- 01_rename_files.py  
|   |-- 02_sort_Airy_RAW.py  
|   |-- 03_split_czi.py  
|   |-- 04_segmentation.py  
|   |-- 04b_control_segmentation.py  
|   |-- 05_quantification.py  
|   |-- 05b_plot_intensity_distribution.py  
|   |-- 06_threshold.py  
|   |-- 06b_plot_thresholds.py  
|   |-- 07_classification.py  
|   |-- 07b_control_classification.py  
|   |-- 08_plot_quantif.py (pas terminé)  
|
```

2 Scripts description

This section describes the role of each of the pipeline scripts.

2.1 config.yaml

This file contains the tunable parameters of the pipeline:

```
1 cellpose:
2   model: "cpsam_v2"          # model cellpose
3   diameter: 120              # average nuclear diameter in pixels
4   flow_threshold: 0.4
5   cellprob_threshold: 1
```

2.2 config.py

This script contains the function to access the configuration parameters of config.yaml:

```
1 def load_config():
2
3   config_path = Path(__file__).parent.parent / "config.yaml"
4
5   with open(config_path, "r", encoding="utf-8") as file:
6       return yaml.safe_load(file)
```

2.3 01_rename_files.py

2.3.1 Objective

This script allows you to rename files to comply with naming conventions. Images naming convention: DiffName_Day_Clone_*; example: LD6_J9_WT_DAPI_PAX3_OLIG3_SOX10_1-Airyscan Processing.

2.3.2 Input

Folder containing the files to rename.

2.3.3 Process

The script performs the following operations:

1. Open a dialogue interface to specify the text to be replaced and the replacement text.
2. Rename all the files from the input folder.

2.3.4 Output

The folder now contains the renamed files.

2.4 02_sort_Airy_RAW.py

2.4.1 Objective

This script sorts the raw images from the airy-scan processed images.

2.4.2 Input

Folder containing the files to rename.

2.4.3 Process

1. Open a window to select the input folder.
2. Create the two output folders: `Airyscan_processed` and `RAW`.
3. Sort the images whose filenames contain "Airyscan" in the `Airyscan_processed` folder and the other images in the `RAW` folder.

2.4.4 Output

- `Airyscan_processed` folder containing the Airy-scan processed images.
- `RAW` folder containing the RAW images.

2.5 03_split_czi.py

2.5.1 Objective

This script splits the CZI images into individual channel images (.tiff).

2.5.2 Input

Folder containing the czi images to split.

2.5.3 Process

1. Open a window to select the input folder.
2. Detect the number of channels in the czi files.
3. Open a dialogue interface to specify the name of each channel.
4. Split the czi images into individual channel tiff images.
5. Save the images in the output folder `images_tiff` with the specified name as a suffix: `"_MarkerName"`.

2.5.4 Output

`images_tiff` folder containing the individual channel tiff images.

2.6 04_segmentation.py

2.6.1 Objective

This script performs nuclear segmentation using the DAPI signal and the Cellpose segmentation model. The parameters (Cellpose model, average nucleus diameter ...) can be configured in the `config.yaml` files.

2.6.2 Input

Folder containing the split images (one channel per image).

2.6.3 Process

1. Open a window to select the input folder.
2. Run Cellpose to segment the nuclei.
3. Save the Cellpose segmentation masks in the output folder: **Segmentation**.

2.6.4 Output

Segmentation folder containing the Cellpose segmentation masks.

2.6.5 04b_control_segmentation.py

This script allows you to color the segmentation masks in order to check that the segmentation is sufficiently accurate. The input is the folder containing the Cellpose segmentation masks, the script applies the LUT *Glasbey on dark* to each images from the input folder and saves the colored masks as jpeg images in a **Control_Segmentation** folder.

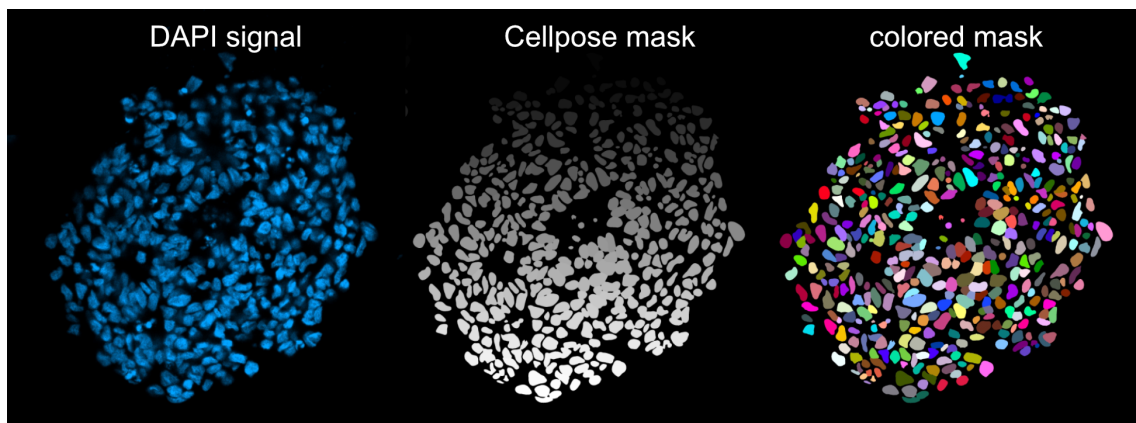


Fig.1 Example of Cellpose segmentation and mask coloration from DAPI signal of a day 9 spinal organoid.

2.7 05_quantification.py

2.7.1 Objective

This script quantifies mean nuclear fluorescence intensity for each marker within each segmented nucleus and exports the results with image and experimental metadata to a CSV file.

2.7.2 Input

First, the folder containing the Cellpose segmentation masks. Then, the folder containing the images (the split images for each channel).

2.7.3 Process

1. Select input folders: segmentation masks and marker images.
2. Detect markers from the first image.
3. Load segmentation masks and marker images for each sample.
4. Measure mean fluorescence intensity for each segmented nucleus and marker.
5. Extract experimental metadata (differentiation, day, clone, condition).
6. Merge measurements across markers and images.
7. Export the results as `quantification.csv`.

2.7.4 Output

A single `quantification.csv` file (cf fig.2).

Image	Differentiation Jour	Clone	Condition	label	mean_OLIG3	mean_PAX3	mean_SOX10
LD6_J9_ISO1_DAPI_F LD6	J9	ISO1	LD6_J9_ISO1	1	301.11786508524835	1223.7203484062268	123.52372127501852
LD6_J9_ISO1_DAPI_F LD6	J9	ISO1	LD6_J9_ISO1	2	1589.1041084962103	599.4168328679697	148.20941364180294
LD6_J9_ISO1_DAPI_F LD6	J9	ISO1	LD6_J9_ISO1	3	1071.7073224525805	849.0280105866784	133.7699602999559
LD6_J9_ISO1_DAPI_F LD6	J9	ISO1	LD6_J9_ISO1	4	1256.03250222618	494.4961412882161	145.60210745028198
LD6_J9_ISO1_DAPI_F LD6	J9	ISO1	LD6_J9_ISO1	5	865.380189725268	368.93224097572994	107.42638905999752
LD6_J9_ISO1_DAPI_F LD6	J9	ISO1	LD6_J9_ISO1	6	325.305913570887	493.22327520849126	74.29681576952237
LD6_J9_ISO1_DAPI_F LD6	J9	ISO1	LD6_J9_ISO1	7	541.066414686825	875.2370410367171	91.53617710583153
LD6_J9_ISO1_DAPI_F LD6	J9	ISO1	LD6_J9_ISO1	8	307.8066298342541	398.1270718232044	39.41988950276243
LD6_J9_ISO1_DAPI_F LD6	J9	ISO1	LD6_J9_ISO1	9	487.9820420728579	484.84025996237386	80.38635197537198

Fig.2 Exemple of a `quantification.csv` output file for three markers: OLIG3, PAX3, SOX10.

2.7.5 05b_plot_intensity_distribution.py

This script generate a density plot of the intensity distributions for each marker. It takes the quantification CSV file as an input, detect marker intensity columns automatically and extract nuclear intensity values for each marker. Then generate the density plots and save them in the output folder: `Plot_Distribution`.

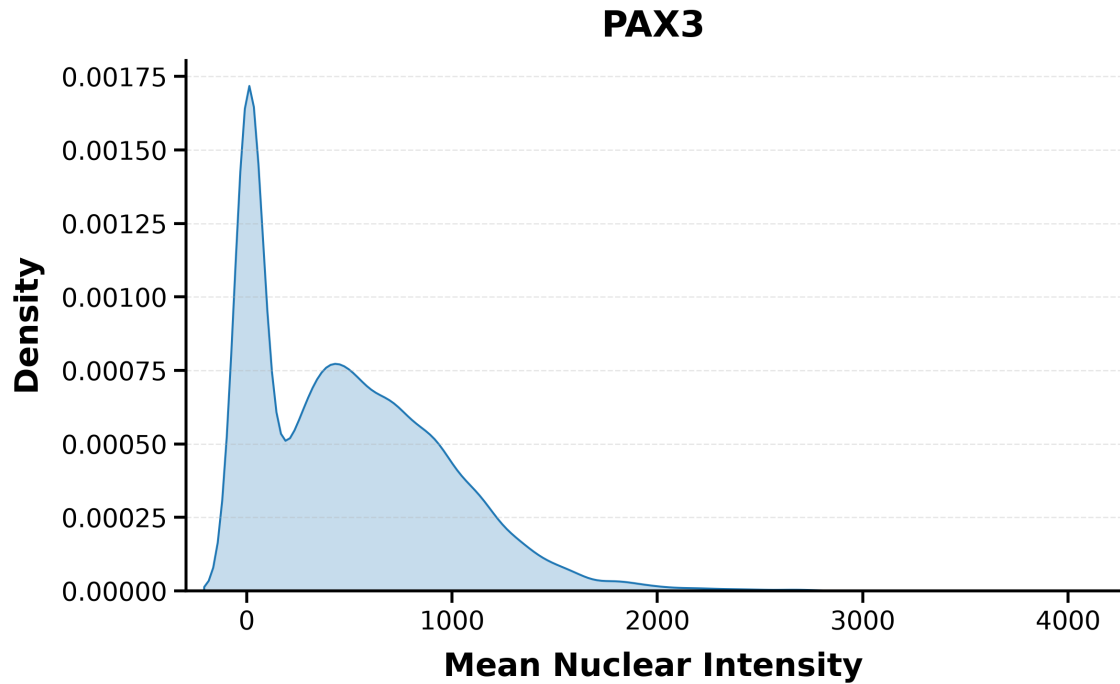


Fig.3 Mean PAX3 nuclear intensity density plot.

2.8 06_threshold.py

2.8.1 Objective

This script displays the intensity distributions for each marker with draggable thresholds to test an initial threshold for distinguishing between positive and negative cell populations.

2.8.2 Input

quantification.csv file containing the mean fluorescence intensity for each segmented nucleus and marker.

2.8.3 Process

1. Select the input file.
2. Automatic marker detection.
3. Display the intensity distribution for each marker with an initial threshold.
4. Interactive threshold selection (calculate the proportion of +/- cells)
5. Save the thresholds as a csv file.

2.8.4 Output

A .csv file containing the thresholds.

2.9 07_classification.py

2.9.1 Objective

This script assigns each nucleus to a mutually exclusive population based on its combination of positive markers according to the specified negative/positive thresholds.

2.9.2 Input

quantification.csv file.

2.9.3 Process

1. Select the quantification CSV file.
2. Automatically detect the markers.
3. The user enters a positivity threshold for each marker.
4. Add the thresholds and classify each nucleus as positive or negative for each marker.
5. Assign each nucleus to a mutually exclusive population based on its combination of positive markers
6. Generate a summary for each image, including total nuclei, marker-positive counts, and counts for each cell population.
7. Save the results as `quantification.csv` and `quantification_summary.csv`.

2.9.4 Output

Update the `quantification.csv` file with the specified threshold for each marker and the classification for each nucleus, and save a `quantification_summary.csv` file containing the cell numbers per image for each combination of markers.

Image	Differentiation Jour	Clone	Condition	total_cells	OLIG3+	PAX3+	SOX10+	Negative	OLIG3_only
LD6_J9_ISO1_LD6	J9	ISO1	LD6_J9_ISO1	505	484	495	0	9	1
LD6_J9_ISO1_LD6	J9	ISO1	LD6_J9_ISO1	761	422	430	0	321	10
LD6_J9_ISO1_LD6	J9	ISO1	LD6_J9_ISO1	575	372	387	0	183	5
LD6_J9_ISO1_LD6	J9	ISO1	LD6_J9_ISO1	683	367	380	0	299	4
LD6_J9_ISO1_LD6	J9	ISO1	LD6_J9_ISO1	213	133	134	0	77	2
LD6_J9_ISO1_LD6	J9	ISO1	LD6_J9_ISO1	289	263	274	0	14	1
LD6_J9_ISO2_LD6	J9	ISO2	LD6_J9_ISO2	437	331	343	0	90	4
LD6_J9_ISO2_LD6	J9	ISO2	LD6_J9_ISO2	951	462	469	0	475	7
LD6_J9_ISO2_LD6	J9	ISO2	LD6_J9_ISO2	448	326	334	0	107	7

Fig.4 Exemple of a `quantification_summary.csv` output file for three markers: OLIG3, PAX3, SOX10. (Only a part of the table is represented).

2.9.5 07b_control_classification.py

This script generates a color-coded classification image showing positive nuclei in green and negative nuclei in red. The user selects the segmentation masks to be visually checked. First, the script loads the quantification table and identifies the analyzed markers. It checks the correspondence between mask labels and quantified nuclei for each image and

retrieves the positivity threshold and classification for each marker. Then, it generates the color-coded classification image for each marker and saves them in the output folder: `Control_Classification`.

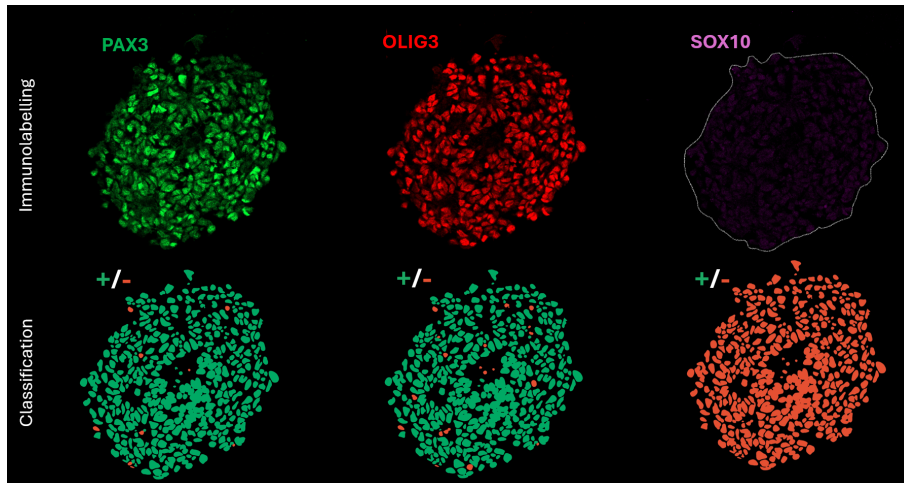


Fig.5 Immunolabelling and classification images for three markers: PAX3, OLIG3 and SOX10.

2.9.6 07c_plot_thresholds.py

This script generates a density plot of nuclear intensity distributions with the threshold separating intensities below and above it (positive and negative nucleus) for each marker. It takes as input the quantification CSV file, detects marker intensity columns, and the associated thresholds automatically. Then, it generates the density plots and saves them in the output folder: `Plot_Thresholds`.

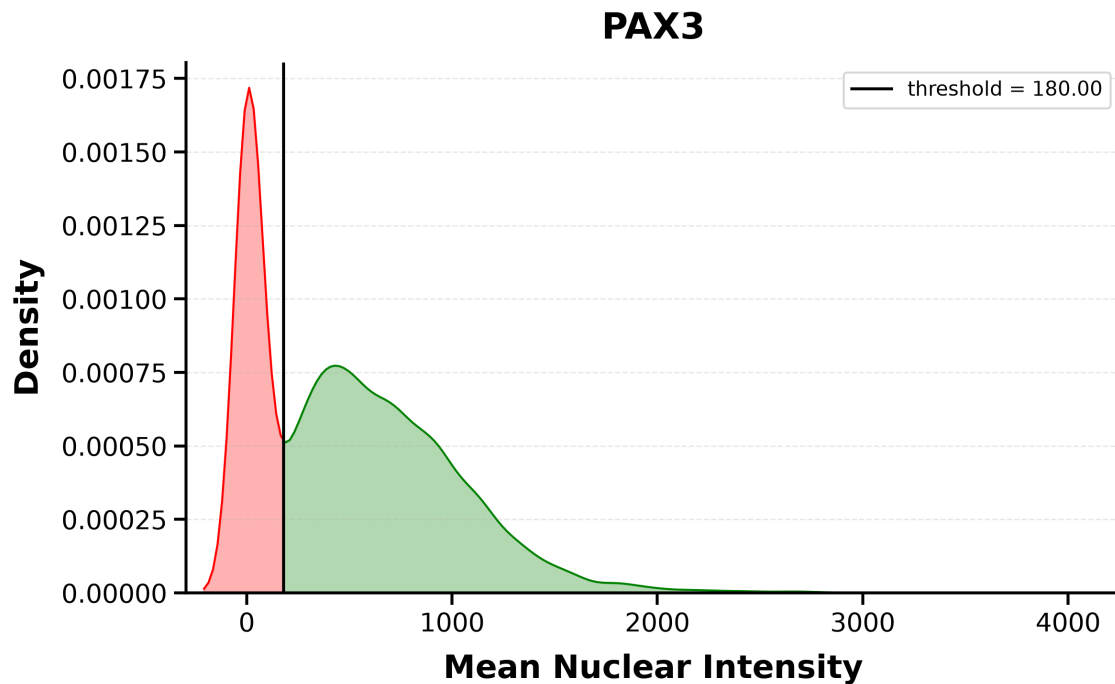


Fig.6 Mean PAX3 nuclear intensity density plot with the selected threshold separating the negative (red) from positive (green) nucleus.

2.10 08_plot_quantif.py

2.10.1 Objective

This script plots the percentage of positive cells for selected marker, with individual measurements for each clone, with each point representing one image.

2.10.2 Input

quantification_summary.csv file.

2.10.3 Process

1. Select the quantification summary CSV file.
2. Specify the marker to analyze and the desired clone order.
3. Calculate the percentage of positive cells for each image from the positive-cell count and total cell count.
4. Plot individual measurements for each clone, with each point representing one image.
5. Add the mean $\pm$ standard deviation (SD) for each clone.
6. Save the graph as a high-resolution PNG in the same folder as the input CSV.

2.10.4 Output

High-resolution PNG graph in the same folder as the input CSV.

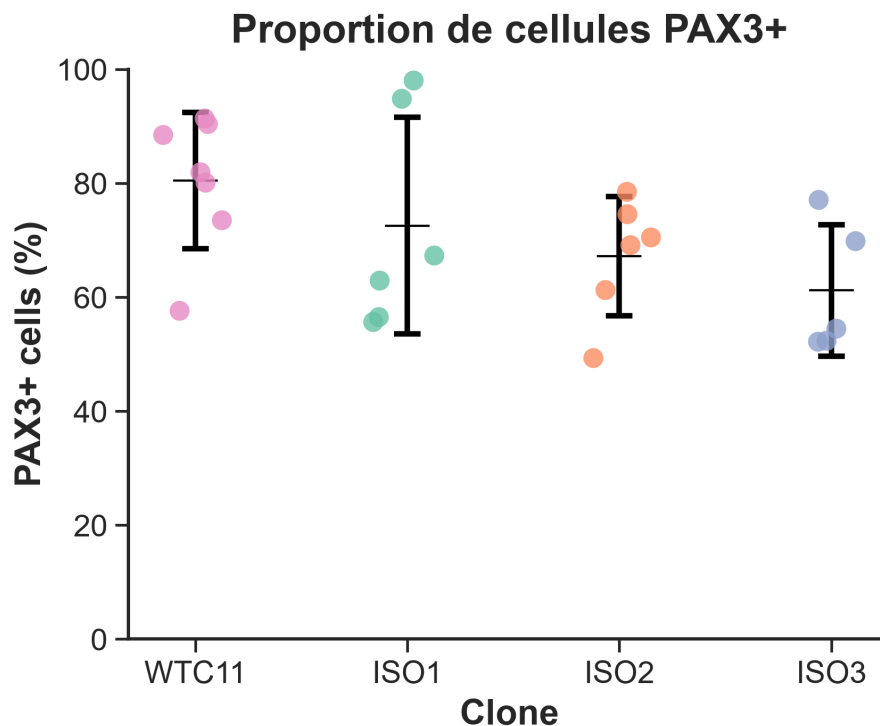


Fig.7 Proportion of PAX3+ cells per clone. Each dot is one image

3 Installation

With Conda :

```
1 conda create -n quantification_pipeline python=3.11
2 conda activate quantification_pipeline
```

Dependencies can then be installed using:

```
1 pip install -r requirements.txt
```

4 How to use the pipeline

4.1 Run a script

A script can be run from the terminal using:

```
1 python Quantification\scripts\04_segmentation.py
```

4.2 Pipeline parameters

The configurable parameters are indicated in :

Quantification\config.yaml

5 Author and contact

Author: Lucas DENIS

Lab: Jacques Monod Insitut, Ribes/Nedelec lab

Contact: lucas.denis@ijm.fr

Last update : [2026-09-25]